

제출 안내 & 산출물 만들기 — AI 웹 서비스 설계 미니 프로젝트

대상: SKALA 4기 비전공자/개발 입문 수강생 (Git, Vue.js, Spring Boot, database 기초를 3주간 학습한 상태)

수준: 입문 — 기술적 완성도보다 설계·문서의 논리적 일관성을 평가합니다.

이 문서 하나에 제출할 **산출물 4종**의

"무엇을 만들어야 하는지"(요구사항·배점·잘한 예/감점 예)와

"어떻게 만드는지"(도구·단계별 순서)를 모두 담았습니다.

한 번에 완벽하게 만들려고 하지 말고, 순서대로 따라 하세요.

 **마감:** 3일차 14:00 — 미준수 시 0점

 **제출 방법:** 산출물 4종을 하나의 zip으로 묶어 제출

1. 제출물 4종 한눈에 보기

#	파일명	답을 내용	만드는 도구	배점
1	{반_이름_프로젝트명}-개요.pdf	프로젝트 기술서 — 서비스 개요·배경, Actor별 요구사항, 전체 화면 UI 흐름(와이어프레임 포함)	워드/한글/파워포인트 + Figma (와이어프레임용)	30점
2	{반_이름_프로젝트명}-API.yml	API 명세 — 화면이 호출하는 전체 API (Method/Path/Request/Response/예외처리)	Swagger Editor (editor.swagger.io)	20점
3	{반_이름_프로젝트명}-DB.dbml	DB 다이어그램 — API 제공에 필요한 모든 데이터 (엔티티·속성·관계)	DBDiagram (dbml.dbdiagram.io)	20점
4	{반_이름_프로젝트명}-발표대본.pdf	발표 대본 — 슬라이드별 내용 + 실제 읽을 발표 스크립트	워드/한글/파워포인트 등	10점

채점 구조 안내: 위 표의 내용 점수 합계는 80점이고, 여기에

납기·제출 완결성 20점(4종을 빠짐없이, 판독 가능한 형식으로, 마감 전 제출)을 더해

총 100점으로 평가합니다. 즉, 형식에 맞춰 4종을 모두 제출하는 것만으로 20점이 확보됩니다.

중요한 원칙 하나: 2번(API.yml)과 3번(DB.dbml)은 `.yml` / `.dbml` 파일 그 자체를 제출해야 합니다. 도구 화면에서 보이는 그림이나 PDF로 변환한 파일이 아니라, 도구가 만들어주는 **코드가 담긴 파일(`.yml` , `.dbml`)을 그대로 저장**하세요.
PDF(1, 4번)는 워드/한글/파워포인트에서 만든 문서를 **PDF로 저장**하면 됩니다.

하나의 기능 흐름으로 통일하세요 (고득점 비결)

4개 파일은 모두 **같은 기능 하나**를 설명해야 점수가 잘 나옵니다. 문서를 만들기 전에 아래처럼 "한 가지 흐름"을 먼저 정하세요.

예시 흐름 (AI 스마트 가게부):

"사용자가 지출을 입력하면 → AI가 지출을 분석해 → 절약 조언을 준다."

- 화면: 거래 입력 화면 → AI 분석 요청 버튼 → 분석 결과 카드
- API: 거래 등록(`POST`), AI 분석(`POST`), 분석 이력 조회(`GET`)
- DB: `users` (사용자), `transactions` (거래), `ai_analysis` (AI 결과)

이 한 가지 흐름이 개요 → API → DB → 발표대본에 **똑같이 반복해서 등장**하게 하세요.
그게 "설계가 논리적으로 연결되었다"는 증거입니다.

2. 시작하기 전에 준비

2-1. 파일명 규칙

모든 파일명은 아래 규칙을 따라야 합니다.

{반_이름_프로젝트명}-{종류}.확장자

예시 — 반 이름이 `판교4`, 이름이 `홍길동`, 프로젝트명이 `AI가게부` 라면 반_이름_프로젝트명 = `판교4_홍길동_AI가게부`

종류	완성된 파일명 예시
개요	<code>판교4_홍길동_AI가게부-개요.pdf</code>
API	<code>판교4_홍길동_AI가게부-API.yml</code>
DB	<code>판교4_홍길동_AI가게부-DB.dbml</code>
발표대본	<code>판교4_홍길동_AI가게부-발표대본.pdf</code>

⚠ 파일명에 띄어쓰기 대신 _ (밑줄)을, 구분은 - (하이픈)을 사용합니다.

예를 들어 판교4 홍길동 AI가게부 처럼 띄어쓰면 안 됩니다.

2-2. zip 파일명 규칙

압축 파일명은 {고유번호}_AI_web.zip 으로 붙입니다 (예: 고유번호가 P100 이면 → P100_AI_web.zip).

고유번호로 제출물과 학생이 매칭되므로 반드시 지켜 주세요.

3. 파일별 작성법 — 요구사항 → 만드는 순서 → 잘한 예/감점 예

3-1. {반_이름_프로젝트명}-개요.pdf — 프로젝트 기술서 (30점)

이 파일에 들어가야 할 것 (3가지)

1. 서비스 개요 및 배경 — 목적, 타겟 사용자, "왜 AI로 해결해야 하는가"(AI 도입 필요성)
2. Actor별 시스템 기능(요구사항) 정의 — 서비스를 사용하는 모든 대상(Actor) 을 빠짐없이 나열하고(예: 사용자, 관리자, 그 외 서비스가 관련된 사람·시스템), 각 Actor가 어떤 기능을 쓰는지
3. 서비스 UI 흐름 — 전체 화면 와이어프레임 — 제공할 화면 전부와 화면 사이의 이동 흐름 (로그인/메인/AI 입력/결과/저장목록 등)

만들기 순서

1단계 — 서비스 개요·배경 쓰기

- 첫 줄에 프로젝트 이름과 "한 줄 요약"을 씁니다.
 - 예: "AI 스마트 가게부: 지출을 기록하면 AI가 지출 패턴을 분석해 절약 조언을 주는 서비스"
- 대상 사용자가 누구인지 씁니다.
 - 예: "월 소비를 관리하고 싶은 사회초년생"
- AI 도입 필요성을 3줄 이상 씁니다. "왜 이 일을 사람 대신 AI가 해야 하는가"
 - 예: "단순 합계만으로는 '왜 식비가 늘었는지' 원인 해석이 어렵다. LLM이 자연어로 해석·조언을 제공할 수 있다."

2단계 — Actor별 요구사항 표 만들기

- 이 서비스와 관련된 모든 대상을 먼저 떠올려 보세요. "누가 이 서비스를 쓰고, 누가 관련되어 있나?"
예: 사용자(일반 회원), 관리자, 그리고 프로젝트에 따라 추가 Actor(예: AI 시스템, 파트너/가맹점, 강사 등).
하나도 빠짐없이 표에 담아야 요구사항이 완전해집니다.
- 아래 모양을 그대로 흉내 내면 됩니다.

| Actor | 역할 | 주요 기능 요구사항 |

| --- | --- | --- |

| 사용자 | 지출을 기록하고 분석을 받는 사람 | 거래 등록 / 거래 조회 / AI 지출 분석 / 분석 이력 열람 |

| 관리자 | 시스템 운영자 | 전체 통계 조회 |

| AI 시스템 | 지출을 분석해 조언을 만드는 프로그램 | 지출 요약 생성 / 자연어 조언 제공 |

3단계 — UI 흐름(와이어프레임) 넣기

- **화면 그림(이미지)을 문서에 삽입**합니다. 와이어프레임은 아래 도구 중 편한 것으로 만드세요.
 - **Figma** (figma.com) — 무료 웹 디자인 도구. 프레임으로 화면을 만들고 화살표로 흐름을 이어주면 전문적인 와이어프레임이 됩니다. 완성 후 각 화면을 **이미지로 내보내기(Export)** 하세요.
 - 워드/한글/파워포인트에서 직접 박스·화살표로 그려도 됩니다.
 - 종이에 손으로 그려서 사진으로 찍어 넣어도 괜찮습니다.
- 화면은 **수업에서 배운 Vue.js 구조**(화면 단위 컴포넌트, 페이지 이동)를 고려해 설계하세요.
- 화면은 **흐름이 보이게** 배치하세요: 로그인 → 메인 → AI 입력 → 결과 → 저장 목록 순서.
- **각 이미지 아래에 텍스트 설명을 반드시 붙이세요.** "이 화면은 무엇이고, 어떤 동작을 하면 다음 화면으로 넘어간다"를 문장으로 적습니다. 이미지만 있고 설명이 없으면 흐름이 제대로 전달되지 않습니다.
 - 예: "거래 입력 화면 — 사용자가 금액과 분류를 입력하고 [저장]을 누르면 다음 화면으로 이동한다."

⚠ **UI 흐름도는 반드시 개요 PDF 안에 이미지로 포함**해야 합니다. 텍스트만 있고 화면 이미지가 없으면 해당 항목의 점수를 인정받을 수 없습니다. 이미지 + 텍스트 설명 **둘 다** 넣으세요.
(Figma 링크만 보내는 것은 인정되지 않습니다. 화면을 **이미지 파일로 내보내 PDF에 넣어야** 합니다.)

잘한 예: 화면마다 이름이 붙고, 사용자가 ① 입력 → ② 저장 → ③ AI 분석 → ④ 결과 보기
흐름으로 이어지는 와이어프레임.

감점 예: 화면 목록만 나열하고 흐름이 없음 / 와이어프레임이 PDF 밖에만 존재 / 이미지 대신 외부 링크만.

4단계 — PDF로 저장하고 파일명 붙이기

- 워드/한글/파워포인트에서 "**다른 이름으로 저장**" → "**PDF**" 를 선택합니다.
- 파일명을 {반_이름_프로젝트명}-개요.pdf 로 저장합니다.

3-2. {반_이름_프로젝트명}-API.yml — API 명세 (20점)

이 파일에 들어가야 할 것

화면을 구현할 때 **호출되는 API 전체 목록**을 OAS(OpenAPI) 형식으로:

- Method(방법: GET/POST) / Path(주소) / Request(보내는 데이터) / Response(받는 결과) / 예외처리 (400·401·404·500 등)
- 이 API들은 이후 수업에서 배운 **Spring Boot(+Spring AI)**로 구현할 것을 전제로 작성하세요.

사용할 도구: Swagger Editor (<https://editor.swagger.io/>)

Swagger Editor는 **왼쪽에는 코드를 쓰고, 오른쪽에는 그 코드가 실제 문서로 보이는** 무료 웹 도구입니다. 코드에 오류가 있으면 바로 표시해 주어서, 문법을 모르는 입문자도 안전하게 만들 수 있습니다.

만들기 순서

1단계 — 사이트 열기

- 브라우저 주소창에 `https://editor.swagger.io/` 입력 후 Enter.

2단계 — 기본 예시 지우기

- 처음 열면 예시 코드(펫스토어)가 채워져 있습니다. **전부 지웁니다.**
- 방법: 왼쪽 코드 창 안을 클릭 → `Cmd + A` (전체 선택) → `Delete` (지우기).

3단계 — 기본 골격 붙여넣기

- 빈 화면에 아래 코드를 복사해서 붙여넣습니다. 이게 API 명세의 "틀"입니다.

```
openapi: 3.0.0
info:
  title: AI 스마트 가게부 API
  version: "1.0"
paths:
```

`title`에는 **자신의 프로젝트 이름**을 넣으세요. 예: `AI 여행 플래너 API`

4단계 — API 하나씩 추가하기

- `paths:` 아래에 API를 한 개씩 추가합니다. 각 API는 **"무엇을 보내고 → 무엇을 받는지"**가 보이도록 씁니다.

아래는 **거래 등록** API의 예시입니다. `paths:` 아래에 그대로 붙여 넣고, 자신의 기능에 맞게 고치세요.

```

/api/transactions:
  post:
    summary: 거래 등록
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              category: { type: string }
              amount: { type: integer }
              tx_date: { type: string, format: date }
    responses:
      '201':
        description: 거래 등록 성공
      '400':
        description: 입력값 오류

```

잘 적었는지 확인하는 방법 (스스로 체크):

- ☐ Method가 목적에 맞는가? (조회= GET , 등록/분석 요청= POST)
- ☐ Path가 화면에서 부르는 주소인가? (예: /api/transactions)
- ☐ Request(요청)에 어떤 데이터를 보내는지 적었나?
- ☐ Response(응답)에 성공(200 / 201)과 오류(400 / 401 / 404 / 500)를 적었나?

5단계 — 오류 확인 (핵심 기능!)

- 왼쪽 코드에 문제가 있으면 **빨간 표시**가 뜹니다. 빨간 줄을 클릭하면 어떤 문제인지 보입니다.
- 오류를 하나씩 고쳐 **빨간 표시가 없어질 때까지** 반복합니다.
- 오른쪽 미리보기가 정상적으로 그려지면 문법은 맞는 것입니다.

6단계 — 파일로 저장하고 파일명 붙이기

- 메뉴에서 **File** → **Save as YAML** (또는 *Download* → *YAML*)을 클릭합니다.
- swagger.yaml 이라는 파일이 다운로드됩니다.
- 다운로드된 파일명을 {반_이름_프로젝트명}-API.yaml 로 **바꿉니다**.

⚠ 반드시 .yaml 파일로 저장하세요. 화면 스크린샷이나 PDF로 저장하면 점수가 인정되지 않습니다.
다운로드한 파일을 메모장으로 열어서 글자가 보이면 올바른 .yaml 파일입니다.

잘한 예:

```

openapi: 3.0.0
info:
  title: AI Smart Ledger API
  version: "1.0"
paths:
  /api/ai/analyze:
    post:
      summary: AI 지출 분석
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                userId: { type: string }
                month: { type: string }
      responses:
        '200': { description: 분석 결과 }
        '400': { description: 입력값 오류 }

```

감점 예: API 목록이 빠짐 / 요청·응답 예시 없음 / 조회인데 POST, 생성인데 GET으로 메서드 오용.

● 3-3. {반_이름_프로젝트명}-DB.dbml — DB 다이어그램 (20점)

이 파일에 들어가야 할 것

API를 제공하기 위해 필요한 **모든 데이터**를 DBML 형식으로 (엔티티·속성·관계):

- 엔티티(테이블) / 속성(컬럼) / 관계 — **PK/FK 명시**, 1:N·M:N 관계, 속성 타입·제약조건
- 약간의 정규화 — 반복되는 데이터는 별도 테이블로 분리

사용할 도구: DBDiagram (<https://dbml.dbdiagram.io/>)

DBDiagram은 **왼쪽에 코드를 쓰면 오른쪽에 ERD(데이터 관계 그림)가 자동으로 그려지는** 무료 웹 도구입니다. 코드가 맞으면 그림이 제대로 보이므로, "내 데이터 설계가 맞나?"를 바로 확인할 수 있습니다.

만들기 순서

1단계 — 사이트 열기

- 브라우저 주소창에 <https://dbml.dbdiagram.io/> 입력 후 Enter.

2단계 — 새 다이어그램 만들기

- 화면의 **Create / New Diagram** 버튼을 클릭합니다.
- (로그인 없이도 만들 수 있습니다. 로그인을 요구하면 "계속하기" 또는 게스트로 진행)

3단계 — 첫 테이블(사용자) 작성

- 왼쪽 코드 창에 아래처럼 테이블을 씁니다.

```
Table users {
  id          bigint [pk, increment]
  email       varchar [not null, unique]
  password_hash varchar [not null]
  nickname    varchar
}
```

해석 도우미:

코드	뜻
Table users { ... }	"users"라는 테이블(표)을 만든다
bigint / varchar	데이터 타입. 큰 숫자 / 문자열
[pk, increment]	이 컬럼이 기본키(PK)이고 자동 증가한다
[not null]	반드시 값이 있어야 한다 (빈 값 금지)
[unique]	중복될 수 없다 (예: 이메일)

4단계 — 기능 테이블과 관계(FK) 추가

- 사용자와 연결되는 기능 테이블을 추가합니다. 연결할 때는 [ref: > users.id] 를 씁니다.

```
Table transactions {
  id          bigint [pk, increment]
  user_id     bigint [ref: > users.id, not null]
  category    varchar [not null]
  amount      bigint [not null]
  tx_date     date [not null]
}
```

- user_id bigint [ref: > users.id] 의 뜻: "transactions의 user_id는 users의 id를 가리킨다" = **한 사용자가 여러 거래**(1:N 관계).
- 오른쪽 화면에 users 와 transactions 사이에 **연결선**이 생기면 관계가 제대로 된 것입니다.

5단계 — 모든 테이블을 빠짐없이 넣기 (정규화 포함)

- 화면/기능에 필요한 데이터가 하나도 빠지지 않게 합니다. "화면에서 보여주는 정보"를 떠올리며 확인하세요.
 - 예: AI 분석 결과를 저장한다면 → ai_analysis 테이블 추가
 - 예: 거래마다 카테고리가 반복된다면 → category 테이블로 분리하고 category_id 로 연결 (정규화)

 **정규화는 "약간"이면 충분합니다.** 반복되는 데이터를 여러 행에 중복해서 넣지 말고
별도 테이블(또는 별도 열)로 분리하세요. 과도한 정규화는 필요 없고,

데이터가 중복되지 않고 관계(FK)로 잘 연결되어 있으면 충분합니다.

6단계 — 파일로 저장하고 파일명 붙이기

- 메뉴에서 **Export** → **DBML** (또는 *Download DBML*)을 선택합니다.
- 다운로드된 파일명을 {반_이름_프로젝트명}-DB.dbml 로 바꿉니다.

⚠ **그림(ERD 이미지)이 아니라 .dbml 파일로 저장하세요.**

Export → PNG/PDF 는 그림이므로 점수가 인정되지 않습니다. 반드시 **DBML 코드** (.dbml 파일) 로 내보냅니다.

잘한 예:

```
Table users {
  id          bigint [pk, increment]
  email       varchar [not null, unique]
  password_hash varchar [not null]
  nickname    varchar
}

Table transactions {
  id          bigint [pk, increment]
  user_id     bigint [ref: > users.id, not null]
  category    varchar [not null]
  amount      bigint [not null]
  tx_date     date [not null]
}

Table ai_analysis {
  id          bigint [pk, increment]
  user_id     bigint [ref: > users.id, not null]
  prompt      text [not null]
  result      text
  created_at  timestamp [default: `CURRENT_TIMESTAMP`]
}
```

감점 예: 화면에 보이는 데이터가 DB에 빠짐 / FK 관계 없음 / 테이블이 1개뿐.

📢 3-4. {반_이름_프로젝트명}-발표대본.pdf — 발표 대본 (10점)

이 파일에 들어가야 할 것

발표할 슬라이드 순서와, 각 슬라이드에서 **실제로 읽을 대본(스크립트)**.

만들기 순서

1단계 — 슬라이드 순서 잡기

아래 순서로 슬라이드를 나눕니다 (7개 정도):

개요 → 배경 → 시스템(전체 구조) → UI → API → DB → 마무리

2단계 — 슬라이드마다 내용 + 대본 쓰기

각 슬라이드는 "내용"과 "발표 대본"으로 구성합니다. 아래 모양을 그대로 따라 쓰세요.

Slide 1: 프로젝트 개요 & AI 도입 동기

- 내용: 한 줄 요약, 목적, 기술 스택.
- 발표 대본: "안녕하세요. 저는 'AI 스마트 가게부'를 소개합니다. 지출을 입력하면 AI가 지출 패턴을 분석해 직접 조언을 주는 서비스입니다."

Slide 2: 핵심 기능 흐름

- 내용: 입력 → 저장 → AI 분석 → 결과 저장.
- 발표 대본: "사용자가 거래를 등록하면 DB에 저장되고, AI 분석 버튼을 누르면 Spring AI가 지출 요약을 분석해 조언을 돌려줍니다."

3단계 — 대본을 실제 말하는 문장으로

- 대본은 실제로 입 밖에 내뱉는 문장으로 씁니다. ("~입니다", "~합니다" 형태)
- 슬라이드마다 대본이 2~3문장이면 충분합니다.

발표 채점은 자비롭게 합니다. 지나치게 길거나 완벽하지 않아도 괜찮으니, 위 순서로 슬라이드와 대본만 갖춰 채워 넣으세요.

💡 여유가 되면 대본 끝에 예상 질문 2개와 답변 2줄을 덧붙이세요.

발표 후 질의응답 대비가 되어 완성도가 올라갑니다.

4단계 — PDF로 저장하고 파일명 붙이기

- 워드/한글/파워포인트에서 "다른 이름으로 저장 → PDF"를 선택합니다.
- 파일명을 {반_이름_프로젝트명}-발표대본.pdf 로 저장합니다.

4. zip으로 묶어 제출하기

1단계 — 4개 파일이 모두 있는지 확인

판교4_홍길동_AI가계부-개요.pdf
판교4_홍길동_AI가계부-API.yml
판교4_홍길동_AI가계부-DB.dbml
판교4_홍길동_AI가계부-발표대본.pdf

2단계 — 파일을 하나의 폴더에 모으기

- 4개 파일을 같은 폴더에 넣습니다.

3단계 — zip으로 압축

- Finder에서 4개 파일을 모두 선택 → **Control** +클릭(또는 두 손가락 클릭) → "**압축(Compress)**".
- 압축된 파일명을 {고유번호}_AI_web.zip 으로 바꿉니다.
 - 예: 고유번호가 P100 이면 → P100_AI_web.zip

⚠ **zip 안에 파일이 바로 보여야 합니다.** "폴더 안에 다시 폴더"가 있으면 안 됩니다.
zip을 열었을 때 4개 파일이 바로 보이는 상태가 올바른 제출입니다.

제출링크: <https://forms.gle/VAGavombJsuiimc99>

✓ 최종 확인 체크리스트

제출하기 전에 아래를 모두 확인하세요.

- ☐ zip 파일명이 {고유번호}_AI_web.zip 형식인가? (예: P100_AI_web.zip)
- ☐ 4종 파일이 모두 들어 있는가? (개요 PDF / API YAML / DB DBML / 발표대본 PDF)
- ☐ 파일명 규칙({반_이름_프로젝트명}-...)을 지켰는가?
- ☐ UI 흐름도(와이어프레임)가 **개요 PDF 안에 이미지 + 텍스트 설명**으로 들어 있는가?
- ☐ API.yml이 .yml 코드 파일인가? (스크린샷·PDF 아님)
- ☐ DB.dbml이 .dbml 코드 파일인가? (ERD 그림 아님)
- ☐ 이미지·PPT·한글·바이너리만 있는 파일로 대체하지 않았는가?
- ☐ 발표대본에 슬라이드마다 **실제 읽을 대본**이 적혀 있는가?
- ☐ 마감(3일차 14:00) 전에 제출했는가? — **미준수 시 0점**

? 막혔을 때 (자주 묻는 질문)

Q1. Swagger Editor에서 빨간 표시가 안 사라져요.

빨간 줄을 클릭하면 원인이 나옵니다. 대부분 들여쓰기(띄어쓰기) 문제입니다. YAML은 "앞의 공백 개수"로 계층을 구분하므로, 위 예시의 공백을 **그대로** 지키는 것이 중요합니다.

Q2. DBML 그림이 이상하게 나와요.

테이블 이름이나 `[ref: > users.id]` 의 `users.id` 가 실제 테이블/컬럼명과 같은지 확인하세요.
오타가 있으면 연결선이 생기지 않습니다.

Q3. 워드/한글 대신 무료 도구를 쓰고 싶어요.

구글 문서(Google Docs)에서 작성 후 "파일 → 다운로드 → PDF"로 저장해도 됩니다.

와이어프레임은 **Figma**(`figma.com`)를 써도 됩니다. 다만 완성한 화면을 **이미지로**
내보내 개요 PDF 안에 넣어야 하며, Figma 링크만 보내는 것은 인정되지 않습니다.

Q4. 시간이 없어요. 무엇부터 해야 하나요?

배점이 큰 **API.yml(20점)**과 **DB.dbml(20점)** 을 먼저 만들고, 개요 PDF(30점)는
"개요 3줄 + Actor 표 + 화면 이미지 2~3장"만 넣어도 큰 점수를 얻을 수 있습니다.
발표대본은 7개 슬라이드 제목 + 각 2문장만 채워도 절반 이상입니다.

✦ **한 줄 정리:** 개요 PDF는 "개요 + Actor 표 + 화면 이미지", API.yml은 Swagger Editor로,
DB.dbml은 DBDiagram으로, 발표대본은 "슬라이드 + 대본"으로 만들어
4개 파일(PDF 30점 · YML 20점 · DBML 20점 · PDF 10점)을
`{고유번호}_AI_web.zip` 으로 묶어 마감 전에 제출하세요.
같은 기능 하나가 4개 파일에 반복 등장하는 것이 가장 중요한 고득점 비결이며,
UI 흐름도는 반드시 개요 PDF 안에 이미지로 포함해야 합니다.